Introduction to Message Passing Interface

Graded Exercise 1

Hai Yu

1 Introduction

The objective of this exercise is to implement and evaluate a matrix computation using the Message Passing Interface (MPI).

The computation involves two double-precision matrices $\mathbf{A}$ with $N = 128000$ rows and $M = 500$ columns, $\mathbf{B}$ with $M = 500$ rows and $K = 1000$ columns, and a vector $\mathbf{c}$ with $K = 1000$ elements. All elements are packed with random values between $[-1, 1]$, using predefined random seeds 10, 20 and 30. The main task is to compute the vector

$$\mathbf{d} = \mathbf{ABc}, \tag{1}$$

followed by its Euclidean norm,

$$\mathcal{N} = \sqrt{\sum_{i=1}^{N} d_i^2}. \tag{2}$$

The computation is parallelized using MPI to distribute the workload across multiple processes with the functions we have leant so far `MPI_*Send/MPI_*Recv`, `MPI_*Bcast`, `MPI_*Reduce`, and `MPI_*Allreduce`.

Performance is evaluated on the AION HPC cluster using 1, 2, 4, 8, 16, 32, 64 and 128 MPI processes on the reservation (`mpi_course`).

Execution times are measured separately for initialization, matrix operations and reduction operations. The resulting measurements are used to analyze speedup and efficiency.

2 Explanation of the Parallelization

The computation is performed in two stages using row-wise matrix decomposition. Both matrices $\mathbf{A}$ and $\mathbf{B}$ are divided into P MPI processes, with any remaining rows assigned to rank 0. This distributes the computational workload while avoiding the storage of the full matrices on worker processes.

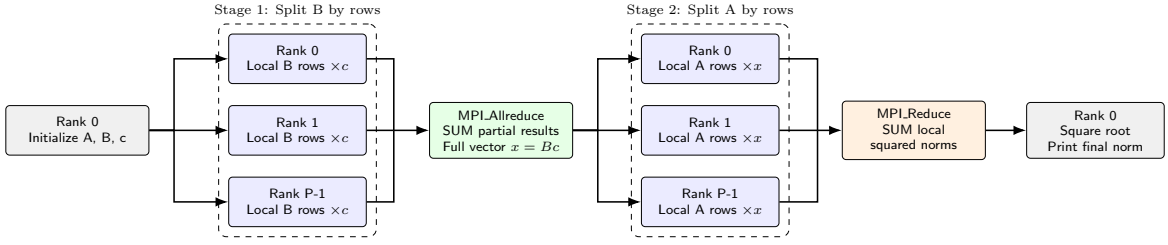


Figure 1: MPI parallel computation workflow for the matrix-vector operations.

Stage 1: Computing x = Bc. Rank 0 initializes the matrices and vector, broadcasts $\mathbf{c}$ using `MPI_Bcast`, and distributes blocks of rows of $\mathbf{B}$ using `MPI_Send` and `MPI_Recv`. Each process computes its local matrix-vector product by function `dgemv`.

The partial results occupy disjoint positions in the intermediate vector. An `MPI_Allreduce` with `MPI_SUM` combines them and makes the complete vector x available to all processes.

Stage 2: Computing $\mathbf{d} = \mathbf{Ax}$. The rows of $\mathbf{A}$ are distributed using the same strategy. Each process computes its local output.

It then calculates the local sum of squares using `ddot`. Finally, `MPI_Reduce` sums these values on rank 0, which computes the Euclidean norm.

Execution times for initialization, matrix operations, and reductions are measured using `MPI_Wtime`.

3 Explanation of the Scaling Results

This is a strong scaling. The results are shown in Figure 2. The initialization phase exhibits no speedup, remaining between 0.73 and 0.84 for multiple processes, because rank 0 initializes and distributes the matrices.

Ranks	Initialization	Operations	Reduction	Total
1	0.281032	0.061716	0.000078	0.342826
2	0.339594	0.030922	0.000137	0.370694
4	0.346939	0.015329	0.016168	0.389399
8	0.342441	0.007655	0.024105	0.386760
16	0.336465	0.003952	0.030285	0.386299
32	0.337626	0.001954	0.006728	0.395449
64	0.341865	0.000941	0.005280	0.406348
128	0.385170	0.000502	0.004575	0.464358

Table 1: Execution times for different numbers of MPI processes (seconds).

In contrast, the matrix operations achieve nearly linear speedup, reaching 122.94 with 128 MPI processes. This demonstrates effective workload distribution and good computational scalability.

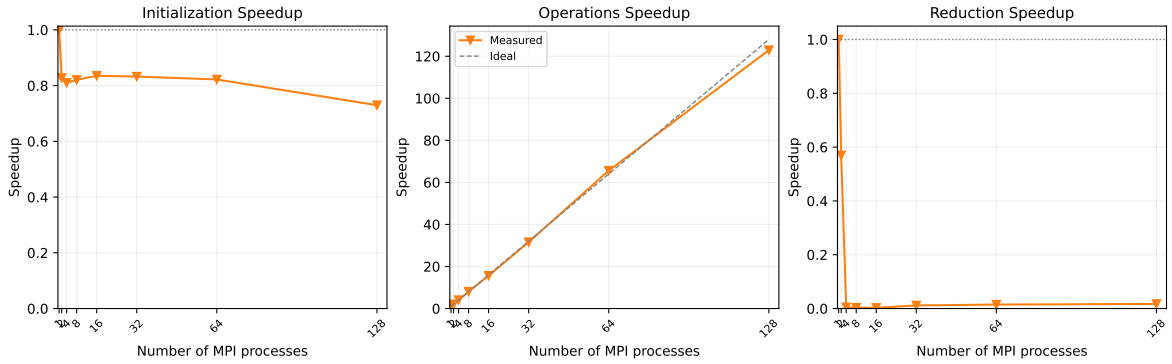


Figure 2: Speedup of initialization, matrix operations, and reduction using MPI.

The reduction phase shows poor scaling. Since the single-process reduction time is extremely small, communication and synchronization overhead dominate as the number of processes increases. Overall, the results demonstrate that the computational workload scales efficiently, while initialization and collective communication limit the performance of the other phases.